PENETRATION TEST REPORT

Kaboom — ICS / OT / Modbus Security Assessment

CONFIDENTIAL

Black-box Assessment

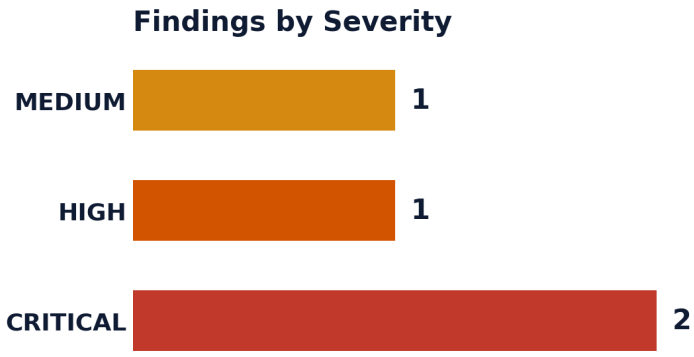
Engagement Type	Black-box assessment
Target	10.48.170.31
Exposed Services	SSH (22), HTTP (8080), Modbus TCP (502)
Category	ICS / OT / Modbus
Status	Completed — Flag Captured
Tester	Sohaeb Gamal Elsayed
Report Date	7 July 2026
Environment	TryHackMe — authorised lab

Table of Contents

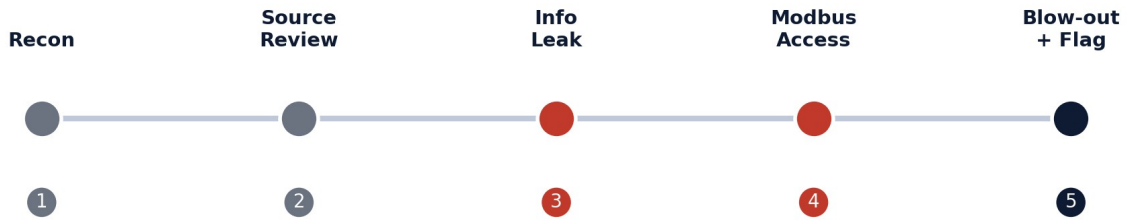
Executive Dashboard

CRITICAL	4	2	1	1
OVERALL RISK	TOTAL FINDINGS	CRITICAL	HIGH	MEDIUM

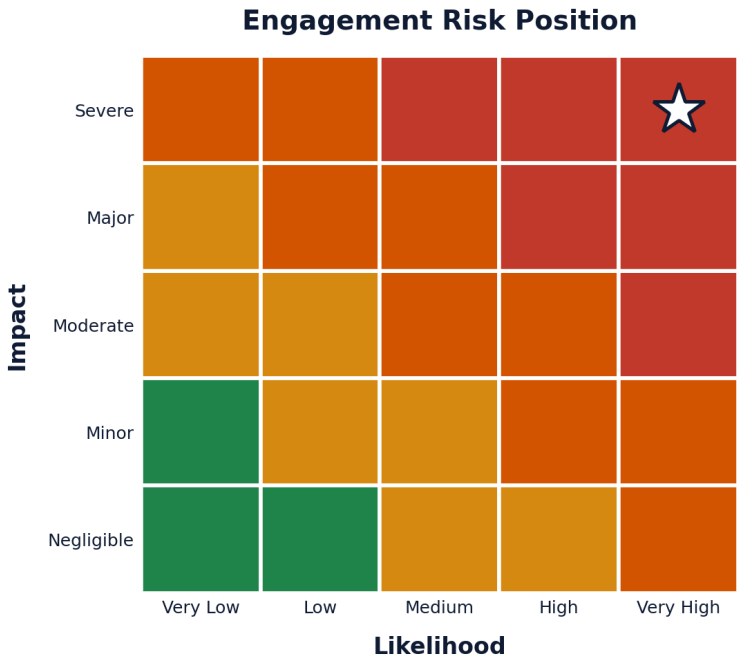
Findings by Severity



Attack Kill Chain



Engagement Risk Position



The engagement plotted at High likelihood / Severe impact: Modbus TCP was reachable with no authentication (high likelihood of exploitation) and a successful attack directly produced a simulated physical safety failure (severe impact).

1. Executive Summary

A black-box penetration test was performed against the ICS/OT host 10.48.170.31 as part of the "Kaboom" engagement. Starting from an unauthenticated position, the tester enumerated exposed web and network services, discovered an unauthenticated status API that leaked an internal secret key and the full Modbus register/coil map, and used that map to issue unauthenticated Modbus writes that over-pressurised the simulated plant and disabled its cooling system — triggering a physical blow-out and capturing the objective flag.

The compromise was enabled by a chain of weaknesses rather than a single flaw. An unauthenticated monitoring endpoint (/api/state) disclosed far more than status information, including a secret key and the exact ICS register/coil addresses. Modbus TCP on port 502 enforced no authentication of any kind, so anyone who could reach the port could both read and write live process values. Finally, the control logic itself had no safety interlock, so a maximum-pressure write and a cooling-disable write could be issued together with nothing to stop them.

> **Overall Risk: Critical**
>
> A single unauthenticated write sequence was sufficient to simulate a physical safety failure. The Modbus exposure (F-02) and missing interlock (F-03) are the most urgent items; the information-disclosure endpoint (F-01) is what made locating the correct registers trivial.

2. Scope & Methodology

2.1 Scope

A single host (10.48.170.31) and two auxiliary hosts discovered during enumeration (10.49.172.137 and 10.49.158.233) were in scope, assessed from an unauthenticated, external (black-box) perspective.

2.2 Methodology

The assessment followed a structured, surface-first methodology, separating enumeration from exploitation:

- **Reconnaissance** — port/service scanning (nmap) and directory/host discovery (gobuster).
- **Source review** — inspection of frontend client-side logic to map API calls and parameters.
- **Attack-surface mapping** — each entry point (login brute force/bypass/SQLi, video parameter, status API, Modbus) recorded as a hypothesis before exploitation.
- **Exploitation** — unauthenticated information disclosure followed by unauthenticated Modbus register/coil writes.
- **Impact validation** — confirmation of the physical/process-level consequence (over-pressure blow-out) and flag capture.

3. Findings Summary

ID	Finding	Severity	Affected Component
F-01	Unauthenticated sensitive information disclosure via /api/state	CRITICAL	/api/state endpoint
F-02	Modbus TCP unauthenticated read/write access	CRITICAL	Modbus TCP :502
F-03	Missing safety interlock allows unsafe state combination	HIGH	ICS control logic
F-04	Information disclosure — server path & OS username	MEDIUM	/api/state endpoint

4. Attack Narrative (Kill Chain)

1. Port scanning revealed SSH (22), an HTTP login page on 8080, and a secondary Werkzeug HTTP service.
2. Directory and host enumeration (gobuster) uncovered two additional hosts: 10.49.172.137 (console/video endpoints) and 10.49.158.233:8080, a session-gated web application.
3. Review of the frontend's client-side source revealed it polled GET /api/state every 5 seconds and rendered a video via GET /video?mode=<value>.
4. The /video?mode= parameter was explored for path traversal / LFI but only exposed already-known modes (default, normal, alarm, maintenance) — a dead end.
5. GET /api/state was found to require no authentication and returned far more than status — a secret key, PIN-locked console access, the server path and OS username, and the full Modbus register/coil map (F-01, F-04).
6. The web login (/login) attack surface — brute force, auth bypass, SQL injection — was tested and blocked; it was not the route to the objective.
7. Modbus TCP on port 502 was connected to directly and required no authentication (F-02); modbus.py was used to enumerate and confirm the register/coil map leaked in step 5.
8. kaboom.py combined write_register(0, 65535) — maxing the pressure/temperature register — with write_coil(15, False) — disabling the cooling/relief valve. No interlock prevented the two writes from being accepted together (F-03).
9. The plant over-pressurised and blew out, and the flag thm{BOOM_BOOM_BOOM} was captured.

5. Detailed Findings

Unauthenticated Sensitive Information Disclosure via /api/state F-01 · CRITICAL

Severity: CRITICAL Component: /api/state endpoint Type: Missing Authentication for Critical Function / Sensitive Data Exposure (CWE-306 / CWE-200)

Description

The frontend polled GET /api/state every 5 seconds with no authentication required. While the endpoint's stated purpose was to report plant status and select a video feed, its response also embedded an internal secret key, a note that console access was PIN-locked but reachable, the server's filesystem path and OS username, and — critically — the full Modbus register and coil map used to control the plant.

Evidence

```
GET /api/state

Response:
{
    "status": "Running",
    "video": "normal"
}

Additional data present in the full response body:
  secret key      : 2Zfe9yNsd8H86ipo7GPr
  console         : PIN-locked, reachable
  server path     : /home/ubuntu/webapp/
  user            : ubuntu
  pressure reg    : address 0
  valve/pump coils: addresses 10-15
```

Impact

This single unauthenticated endpoint handed over everything needed to attack the OT layer directly — an attacker never needed to authenticate to the web application at all to obtain the ICS register/coil map used in F-02 and F-03.

Remediation

- Require authentication on all status/monitoring endpoints, not only on control endpoints.
- Strip secrets, internal paths, and register/coil mappings from any response reachable by an unauthenticated client.
- Rotate the exposed secret key and audit what other 'harmless' endpoints return.

Modbus TCP Unauthenticated Read/Write Access F-02 · CRITICAL

Severity: CRITICAL Component: Modbus TCP :502 Type: Missing Authentication for Critical Function (CWE-306)

Description

Modbus TCP was reachable on port 502 with no authentication of any kind. Holding registers and coils could be freely read and written by any client able to route to the port, including the pressure indicator (register 0) and the six valve/pump control coils (addresses 10-15).

Evidence

```
from pymodbus.client import ModbusTcpClient

c = ModbusTcpClient('10.48.170.31', port=502)
c.connect()

# Sweep holding registers
rr = c.read_holding_registers(address=0, count=20, slave=1)
print(rr.registers)

# Sweep coils
rc = c.read_coils(address=0, count=20, slave=1)
print(rc.bits)

c.close()
```

Impact

Any network-reachable client can both observe and manipulate live process values — pressure, valves, and pumps — with no credential, session, or source restriction of any kind.

Remediation

- Segment IT/OT networks so Modbus TCP is never reachable from general-purpose or DMZ network segments.
- Where the protocol/hardware allows, deploy Modbus security extensions or place an authenticating gateway/proxy in front of the PLC.
- Apply strict firewall allow-listing so only known engineering workstations can reach port 502.

Missing Safety Interlock Allows Unsafe State Combination F-03 · HIGH

Severity: HIGH Component: ICS control logic Type: Improper Enforcement of Behavioral Workflow / Missing Interlock (CWE-841)

Description

Even given the Modbus exposure in F-02, a well-designed control system would refuse to accept a combination of writes that is physically unsafe. Here, the pressure/temperature register could be driven to its maximum value at the same time the cooling/relief coil was disabled, with nothing in the control logic to reject or flag the combination.

Evidence

```
client = ModbusTcpClient("10.48.170.31", port=502)
client.connect()

# Step 1: Spike the temperature to max
client.write_register(0, 65535)

# Step 2: Kill the cooling system
client.write_coil(15, False)

# Result: plant over-pressurises — no interlock catches
# the combination of max pressure + cooling disabled
```

Impact

A single short script was sufficient to simulate a physical safety failure and trigger a plant blow-out, demonstrating that the control logic itself — not just network access — is a point of failure.

Remediation

- Implement interlocks at the PLC/control-logic layer that reject unsafe combinations of setpoints regardless of how the write was issued.
- Add independent, hard-wired safety instrumented systems (SIS) that are not reachable or overridable via the same network path as the process control writes.
- Alarm and log any write that approaches unsafe thresholds, independent of network-layer controls.

Information Disclosure — Server Path & OS Username F-04 · MEDIUM

Severity: MEDIUM Component: /api/state endpoint Type: Information Exposure (CWE-200)

Description

Alongside the secret key and Modbus map, the same unauthenticated /api/state response disclosed the application's server-side filesystem path and the OS-level username running the service.

Evidence

```
server path : /home/ubuntu/webapp/
user        : ubuntu
```

Impact

This information narrows follow-on attacks (e.g. path-based file access, targeted credential attacks against a known username) and should not be exposed to unauthenticated clients.

Remediation

- Remove internal path and account details from any client-facing response.
- Run web services under a dedicated low-privilege account rather than a named personal/admin account.

6. Post-Exploitation

Exploitation of the Modbus write path (F-02, F-03) resulted in a simulated physical process failure — the plant over-pressurised and blew out — and the objective flag was captured:

```
thm{BOOM_BOOM_BOOM}
```

Note on documentation completeness

The scenario's narrative references a stealth implant in the DMZ during the diversion. That

> *secondary objective was outside the scope of this technical assessment, which focused on the ICS/Modbus attack path; it is noted here for completeness and should be scoped separately if required.*

7. Remediation Summary

ID	Action	Priority
F-01	Require auth on status/monitoring endpoints; strip secrets and register maps from responses.	CRITICAL
F-02	Segment IT/OT networks; restrict or gateway Modbus TCP :502 access.	CRITICAL
F-03	Add control-logic interlocks and independent safety instrumented systems.	HIGH
F-04	Remove internal path/username disclosure; run services under least-privilege accounts.	MEDIUM

Appendix A — Services & Testing Performed

Discovered Services

Port	Service	Version / Notes
22	SSH	OpenSSH 9.6p1
8080	HTTP	Werkzeug → login_page
502	Modbus TCP	No authentication; pressure at register 0, valves/pumps at coils 10-15
—	HTTP	Werkzeug httpd (secondary host)

Testing Performed (Including Ruled-Out Paths)

The following avenues were considered and ruled out. Recording them is part of mapping the attack surface — eliminating a path is as valuable as confirming one.

Avenue Tested	Result
/video?mode= — path traversal / LFI / mode enumeration	No further leakage beyond known modes; dead end
/login — brute force	Attempted; not the route to the objective
/login — authentication bypass	Blocked
/login — SQL injection	Blocked
/login — response timing / user enumeration	Noted, not confirmed as a viable path

Appendix B — Attack Path Summary

```
IP (10.48.170.31)
 └─ scan
     ├─ nmap → SSH(22), HTTP(8080 login), HTTP(Werkzeug)
     ├─ gobuster → 10.49.172.137 (console/video), 10.49.158.233:8080 (webapp)
     └─ source code review
         ├─ /video?mode=… → dead end
         └─ /api/state (unauthenticated) → F-01 / F-04
             ├─ leaked internal secret key
             ├─ console reachable (PIN-locked)
             ├─ disclosed server path + username
             └─ Modbus register/coil map (pressure=0, valves=10-15)
                 └─ Modbus TCP:502 (no auth) → F-02
                     ├─ modbus.py — enumerate/confirm registers & coils
                     └─ kaboom.py — write_register(0,65535) + write_coil(15,False)
                         └─ no interlock (F-03) → blow-out
                             └─ FLAG: thm{BOOM_BOOM_BOOM}
```

Meanwhile, the web login (/login) attack surface — brute force, auth bypass, SQL injection — was explored but blocked; it was not the route to the objective. The actual path was: unauthenticated info-leak on /api/state → unauthenticated Modbus write → missing safety interlock.